the term *.NET 5+* to refer to .NET 5, .NET 6, and .NET 7, and the term *.NET Core* to refer to previous versions.

.NET Core (and its successor, .NET 5+) employs many of the same APIs as .NET Framework but is more modular. It implements a different set of features from those in .NET Framework, with the goal of providing a simpler programming model and modern APIs. It's a separate platform rather than a fork of .NET Framework, though it uses similar code for many of its APIs.

NOTE If you'd like to learn more about the .NET ecosystem, you can read two posts on my blog: "Understanding the .NET ecosystem: The evolution of .NET into .NET 7" (http://mng.bz/Ao0W) and "Understanding the .NET ecosystem: The introduction of .NET Standard" (http://mng.bz/ZqPZ).

The benefits and limitations of ASP.NET

ASP.NET Core is the latest evolution of Microsoft's popular ASP.NET web framework, released in June 2016. Previous versions of ASP.NET had many incremental updates, focusing on high developer productivity and prioritizing backward compatibility. ASP.NET Core bucks that trend by making significant architectural changes that rethink the way the web framework is designed and built.

ASP.NET Core owes a lot to its ASP.NET heritage, and many features have been carried forward from before, but ASP.NET Core is a new framework. The whole technology stack has been rewritten, including both the web framework and the underlying platform.

At the heart of the changes is the philosophy that ASP.NET should be able to hold its head high when measured against other modern frameworks, but existing .NET developers should continue to have a sense of familiarity.

To understand why Microsoft decided to build a new framework, it's important to understand the benefits and limitations of the legacy ASP.NET web framework.